

FRED TALKS

19th September 2026

CONTENTS

What does "playing politics" mean for software engineers?

SEANGOEDECKE.COM RSS FEED · 2490 WORDS

It's-a Me, Agentic AI

HAN, NOT SOLO · 2362 WORDS

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 1053 WORDS

Codex CLI Release: 0.155.1

CODEx CHANGELOG · 48 WORDS

The absolute best AI tools to use in September 2026

CASSIDY WILLIAMS · 2 WORDS

What does "playing politics" mean for software engineers?

SEANGOEDECKE.COM RSS FEED · 14 JUL 2026 · [SOURCE](#)

Software engineers are often told to “start playing politics”, but most engineers have no idea what that means.

Their reference point for “playing politics” comes from fiction like Game of Thrones. Are they supposed to raise an army and depose the CEO, or poison each other at team lunch? Should they book Zoom calls with each other and plot schemes? All of that is obviously ridiculous. In terms of Game of Thrones, software engineers are not lords and ladies. We’re the soldiers and workers of the realm. So you should think about “playing politics” in the way a castle guard would, not one of the major players.

The castle guard are not going around poisoning people or forming coalitions between the great powers. They are largely keeping their heads down. But in order to do that, they have to stay aware of the political currents, or they’re liable to do something catastrophically stupid: for instance, making an enemy of a powerful courtier, or arresting somebody who’s on an important mission for the king.

Given that, the basic principles of playing politics are something like this:

- Be aware of who’s powerful and who’s not
- At all costs, avoid making powerful enemies
- Help powerful people as best you can
- Make sure they know you’re helping them (without annoying them)

Be aware of who's powerful and who's not

As a software engineer in a large company, **you will not be a powerful person**. Powerful people are typically in senior management: VPs, directors, and so on¹. However, not everyone in senior management is powerful. Some are killers who have the active support of the CEO, while others are confused incompetents.

How do you know which is which? If someone is clearly ferociously competent, they're always going to have *some* power, since upper management tend not to ignore useful tools. But you can't rely on competence as your only guide. Some managers are powerful for other reasons: they're friends with the CEO, or they have strong relationships with other groups like legal or sales, or they're simply willing to do whatever upper management wants done.

One signal is who's leading the important projects. Read your CEO or CTO's internal updates and pay attention to the projects that are called out by name. Organizations tend to give key tasks to trusted lieutenants. If a manager is leading an area that's never under the spotlight, they probably don't have enough clout.

Another signal is hiring. Is a manager's team growing or shrinking? Particularly post-ZIRP, headcount is a rare and precious resource. A manager who's able to get it is likely a powerful manager, or at least is reporting to a powerful director or VP (which often amounts to the same thing).

At all costs, avoid making powerful enemies

First, you should try not to make any enemies at all. Most software engineers who get "playing politics" wrong do it by needlessly alienating people: by being rude, unhelpful, abrasive, making non-technical people feel stupid, and so on. This post isn't really about that. I'm assuming that you can figure out how to be a generically pleasant person on your own.

However, **competent software engineers will make some enemies**. If you're out there making projects happen, some people aren't going to like the way you do it, and won't be a fan of any compromise you offer. I wrote about this in *Big tech engineers need big egos*: the only way to avoid making enemies is to change nothing, but that's incompatible with doing the job.

Given that, be selective about *which* enemies you make. If you're making a technical decision that's either going to require work from team A or team B, and neither team wants to do it, you should try to pick the team with the least political cover. If you need a powerful VP's team to do something they won't like, try to be maximally respectful about it: get that team's core engineers on-side if you can, or book a meeting with the powerful manager and explain the situation, or (better yet) ask the powerful manager sponsoring your project to go and talk to the other VP for you. (If you don't have a powerful manager like this, consider abandoning your project).

Give way to powerful managers when at all possible. Every so often you really do have to stand your ground — if the system will truly collapse otherwise, or a major customer will have an incident, or if the technical decision really is entirely bone-headed — but almost all cases are not like this. The best advice I've ever gotten about playing politics came from a manager I worked with long ago²:

| This is not the hill you want to die on.

When I'm about to pick a fight or say something argumentative, and I'm not 100% convinced it's necessary, I ask myself: is this the hill I want to die on? And it never is.

The three rules about disagreeing with powerful people are:

- Make sure you do it in private
- Be polite
- When they overrule you, stop arguing immediately

Disagreeing in private rarely hurts, if you follow these rules. In fact, it can help. If you can manage to disagree with a manager, get overruled, and then follow their plan without complaining, that can be the best way to gain a powerful friend. But if they think you're going to keep griping about it, or worse still, complain to the rest of the team and foment some kind of rebellion, there's no quicker way to make a powerful enemy.

If you have powerful enemies at a company (for instance, the CTO or an influential VP doesn't like you), **quit**. It's really that bad. I have never seen this situation turn itself around, except in the very rare case where the CTO or VP is already looking for greener pastures and jumps ship. You cannot recover the situation: they have no incentive to give you the chance to change their mind, and they have almost unlimited ability to screw you on promotions, raises and layoffs.

That's why this piece of advice is second in the list. If you aren't helpful or if your contributions are invisible, you can work on that and fix it. But if you've made powerful enemies, you're done for.

Help powerful people as best you can

Just as it's fatal to make powerful enemies, it's very useful to make powerful friends. How can you do this? Remember you're a palace guard, not a great lord: you make friends **by doing your job**. However, you can choose to do your job a little more proactively and diligently when you're doing it for someone with political clout.

One obvious application of this principle is that **you should answer Slack messages from powerful people immediately**. If you see an ordinary Slack question pop up while you're doing some task, it's okay to get to it when you get to it. In fact, it's ideal *not* to respond to all questions immediately, so you don't set unreasonable expectations (and so you don't seem like you're sitting around doing nothing). But when a VP comes in with a question, don't make them wait: answer the question immediately. If the question requires research, send a "let me look into that right now" message, then do the research. This is the easiest way to get a reputation for being helpful³.

Another way to do this is to **lean in on important projects**. Suppose you do ten projects in a year. Eight of them are normal, low-priority projects, and two of them are high-profile (say, finishing some big feature before your company's yearly conference). It's a mistake to allocate your effort equally to all ten. I wrote about this at length in *Doing nothing at work*: you should be operating at 80% capacity (or less), so you can then ramp up to 120% when it really matters.

Pay attention to the narrative that powerful people are trying to push. Here are some potential narratives:

- We've had a lot of turnover and reorgs lately, but we're all starting to pull together as a team now
- Isn't it great how focused we all are on reliability work after last month's incident?
- The conference this week is the most important thing, so we're all being very careful not to break anything
- We're an AI-forward team that's looking for the best ways we can leverage LLMs into our team processes
- Although this project had a rocky start, we're now all aligned on the way forward

You don't necessarily have to jump in and start cheerleading, but you should at least not do anything that you know is going to make the narrative look weak. For example, on that last point, it's foolish to openly argue that the project really was fine all along. Bring it up privately, not publicly, or you risk ruining some clever piece of propaganda that the manager in question is trying to push on the rest of the organization⁴.

Finally, an underrated way to help powerful people is to offer them social support and information. Slack messages and planning emails might seem unimportant to you, but powerful people often live in that environment: their primary tool is writing messages like these, just like your primary tool is writing code. Reading and responding (in a supportive way) to these messages is something that most engineers don't bother to do, but it goes a long way.

Likewise, dropping a senior manager a line now and then (say, a heads-up that a particular project landed successfully, or that you got good metrics about some feature) is surprisingly helpful. Senior managers live in an information-poor environment: for them to learn something about a team's work, that information has to bubble up through several layers of interpretation and summary. In my experience, they're appreciative of being drip-fed the occasional piece of information, so long as you keep it brief and relatively rare.

Make sure they know you're helping them

If you're directly responding to a VP's Slack messages or DMing them information, they know you're the one doing it. But if you're just doing your job and working hard on projects they care about, they might not notice. **Being invisible is probably the most common way engineers fail at playing politics.**

Fortunately the fix is simple: tell people what you're doing. If you fix an important bug for a launch, write a message in that launch's Slack channel saying "hey, I just fixed this bug". What if you don't like bragging? Get over it. You have to be comfortable publicly telling people what you've done. You should also keep a brag document so you can repeat all of this at review time.

Another, subtler way to do this is to gain the trust and respect of the powerful engineers in your area. Senior managers will always have a few trusted engineers they rely on to assess technical questions. They will ask those engineers what they think about you, and will broadly trust those answers. The good news is that if you're competent and useful, those engineers will already value you, so you don't have to do anything special: just be good at your job.

Technical power

Is playing politics all about sucking up to senior managers? Basically, yeah. A less cynical way to describe it would be "aligning with the values of the company". If you think your company is doing good things, you should want to do that anyway! In any case, what that comes down to is figuring out what the people in charge want, giving it to them, and making sure they see you doing it. However, there's still some scope to get what *you* want out of the deal.

I said earlier that software engineers do not wield organizational power. However, that doesn't mean you're powerless. Technical ability is a source of real power, if a delicate and unreliable one. The movers and shakers in tech companies are utterly dependent on technical people to implement their vision and to give them clear answers about the system.

There are many subtle ways you can leverage this. One I wrote about in *How I influence tech company politics as a staff software engineer* is to wait until important people at the company want to do something (say, improve reliability), then offer them a technical plan that does it your way. Another one is to become so useful that you're actively in demand to lead projects, and then run the project how you want.

You probably won't be able to change the company's grand strategy. But how that strategy is *implemented* has a lot of specific technical detail, and you can put yourself in a position to decide on those details.

Conclusion

Playing politics isn't about plotting and scheming, and it isn't just about being a friendly, likeable person (although that helps). It's about figuring out how your company actually operates: who makes the decisions, who gets consulted, what behavior gets rewarded, and so on. The most basic way to do that is to **figure out who is powerful, get out of their way, and (if you can) help them get what they want.**

1. Obviously the exact titles depend on your company. One person I'm deliberately leaving out is your own manager. In general don't think your relationship with your own manager counts as "playing politics": that's just you getting along with another human being. An exception to that is if you report directly to a powerful director or VP.

↩

2. Ironically, this manager struggled to take his own advice.

↩

3. Note that you actually have to be able to answer their question accurately in order to do this. If you're not competent enough to be useful to powerful people, you will struggle to befriend them.

↩

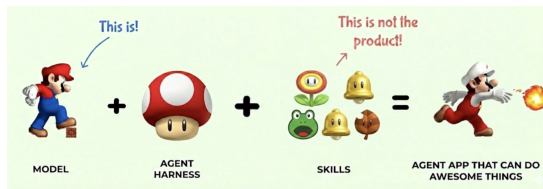
4. For instance, maybe the CEO is convinced that the project was in bad shape because of something he heard, and the manager in question knows it's easier to sell "yes, but we turned it around" than "no, you misunderstood, everything was always fine". If you complicate that process, you risk the CEO thinking that the project is still bad and cancelling it.

↩

It's-a Me, Agentic AI

HAN, NOT SOLO · 01 MAR 2026 · [SOURCE](#)

Agentic AI is a fairly recent development that combines reasoning ([OpenAI, 2024](#)) tool use ([Schick et al, 2023](#)) in the same AI model. But an agentic AI system is not just the model, but also the harness, the environments, the tools, rewards, evaluations, and all of the infrastructure to support it. In this post, let's use the top grossing franchise, Mario, to tell this story for understanding how agentic AI models are developed, how agent harnesses work, and how reinforcement learning ties everything together. If you survived World 8-4 as a kid, you already have the intuition for building agentic AI systems.



alt text

Small Mario is the **base pretrained model**. He's just come out of pretraining on a massive corpus of platform game physics. He can walk. He can jump. He can move left and right. These are his base capabilities, the raw knowledge compressed from the training data.

But Small Mario is fragile. One hit from a Goomba and he's dead. He can't break bricks. He can't take damage. He has potential, but he's not yet useful for anything beyond the most trivial tasks.

This is your base LLM fresh off pretraining. It has absorbed enormous amounts of knowledge, it can do next-token prediction, and it can sort-of follow instructions. But ask it to do anything real, reliably, in production, and it falls apart on the first obstacle. One Goomba and it's game over.

The Super Mushroom: Model Harness

Then Mario finds the **Super Mushroom**. He doubles in size. He can now break bricks. He can take a hit without dying. He goes from fragile to capable.

The Super Mushroom is the **model harness**, the entire infrastructure and post-training pipeline that transforms a base model into something production-ready. This includes:

- **Instruction tuning** (RLHF, DPO, etc.) to make the model actually follow directions
- **System prompts** that define personality and constraints
- **Safety guardrails** so it doesn't run off cliffs
- **Memory and context management** so it remembers where it's been
- **Tool-use training** so it knows power-ups exist and how to grab them

Without the Super Mushroom, Mario is a liability. With it, he's a platform for greatness. Similarly, without the model harness, a base LLM is a research artifact. With it, it's a product.

The Super Mushroom doesn't change WHO Mario is. It changes what he can SURVIVE. The model harness doesn't change the model's core knowledge. It changes what the model can handle in production.

Now here's where it gets interesting. Super Mario can pick up **power-ups** that give him entirely new capabilities:

Power-Up	Mario Ability	Agent Equivalent
Fire Flower	Throw fireballs to defeat enemies at range	Code execution - reach out and run code to solve problems the model can't solve with text alone
Frog Suit	Swim through water levels with ease	Web search - navigate and retrieve information from an environment the model wasn't trained on
Raccoon Leaf	Fly and reach otherwise inaccessible areas	

File system access - explore and modify the codebase, reach files the model couldn't otherwise touch

Hammer	Throw hammers, defeat almost anything	Shell/terminal access - the most powerful tool, can interact with the full system
Suit		
Star	Temporary invincibility, blow through everything	Extended thinking/reasoning mode - brute force through complex problems at higher compute cost
Cape	Sustained flight and spin	MCP servers - sustained, extensible access to external services and APIs
Feather	attack	

Each power-up doesn't replace Mario's core abilities. Mario still walks and jumps. The power-ups **extend** what he can do. A Fire Flower Mario can still jump on Goombas, but now he can also shoot fireballs at Piranha Plants hiding in pipes.

This is exactly how agent tools work. The LLM still does what LLMs do: reasoning, language understanding, and planning. Tools extend the model's reach into environments it can't operate in alone. An LLM can't execute Python by itself, just like Mario can't throw fireballs without a Fire Flower. But give it the right tool, and suddenly the problem space opens up.

And critically, **Mario has to learn WHEN to use each power-up**. Frog Suit is amazing in water levels, useless on land. Fire Flower is great against Goombas, pointless against Thwomps. The model needs to learn tool selection, knowing which tool to reach for in which context. This is one of the hardest parts of building agentic systems.

One power-up deserves special attention: the **Star**. When Mario grabs a Star, he becomes invincible. He plows through Goombas, Koopa Troopas, Piranha Plants, everything in his path just disintegrates. Nothing can stop him. This is not dissimilar to having an engineering manager who's really good at clearing organizational blockers for their engineers. The Goombas and Piranha Plants of bureaucracy, cross-team dependencies, access requests, and priority conflicts just melt away. Star power is temporary and expensive, but when you need to blast through a critical path, nothing else comes close.

The Mushroom Kingdom: Environments and Tasks

Now let's talk about the world Mario operates in. Every level in the Mushroom Kingdom is an **environment**, and every environment is composed of the same building blocks:

Level Element	Environment Equivalent
Bricks	

	Structured data, APIs, databases that can be broken open to reveal resources
? Blocks	Unknown information sources, sometimes containing exactly what you need
Pipes	Entry points to sub-tasks, function calls, or deeper exploration
Coins	Incremental progress signals, intermediate rewards
Goombas	Common obstacles, predictable errors, edge cases
Koopa Troopas	Recyclable obstacles, errors whose solutions can be reused as tools
Piranha Plants	Traps hiding in seemingly safe passages, hallucination triggers
Vines	Hidden paths to bonus areas, unexpected shortcuts
Clouds / Platforms	Abstractions and scaffolding that support traversal
Pits	Catastrophic failures, unrecoverable errors

Every level remixes these elements differently. World 1-1 is simple, a few Goombas, some bricks, a clear path to the flag. World 8-4 is a maze of pipes, hidden paths, and a boss fight with Bowser. Same building blocks, radically different difficulty.

Throughout each level, Mario interacts with the environment as **tool use**. He enters pipes to warp from one place to another, the equivalent of an API call that transports you to an entirely different context. He bumps ? Blocks from below to discover power-ups, new agent skills materializing from the environment when you know where to look. He breaks bricks to clear paths or reveal hidden rewards, structured data yielding its value when you apply force in the right direction. He stomps on a Koopa shell and kicks it forward, turning an obstacle into a projectile that clears a line of Goombas, repurposing error outputs as inputs to solve downstream problems. The environment isn't just a backdrop. It's a toolbox.

Having an environment is not enough. We need to define **what we want to achieve** from playing the game. These are the **tasks**.

Mario can play the same level with completely different objectives. Complete the level. Complete the level in the shortest amount of time. Get the highest score. Collect the most coins. Accumulate the most 1-up lives. Find all the hidden rewards. Stomp on every last Goomba. Each objective produces a fundamentally different play style from the same environment.

This is exactly the task definition problem in agentic AI. The same model operating in the same environment can be pointed at wildly different goals. “Summarize this codebase” and “refactor this codebase” use the same files, the same tools, the same context, but they require entirely different strategies. The task is what transforms an environment from a sandbox into a mission.

The Flagpole: Evaluations and Reward Modeling

At the end of every level, there's a **flagpole**. Mario jumps on it, pulls down the flag, and receives a reward. The higher he grabs the flag, the bigger the reward. Some levels end with a boss fight against Bowser, where the reward is freeing a Toad (or eventually, the Princess).

This is the **reward signal** in reinforcement learning. But here's the critical question: how do we actually **measure** how well the game was played? This is **evaluation**, or **reward modeling**, and it is where the machine learning engineering discipline really shines. (or research engineer, applied AI engineer.)

The evaluation could be the raw score. It could be the number of 1-ups gained. Coins collected. Goombas stomped. Time remaining. Different paths discovered. Most frequently, it is a **combination** of some or all of the above, weighted and balanced against each other. Do we reward Mario more for speed or for thoroughness? For survival or for aggression? For finding secrets or for staying on the critical path?

Evaluation Metric	Mario Measure	Agent Measure
Speed	Time remaining on the clock	Task completion latency
Score	Points accumulated	Overall output quality
Collection	Coins gathered	Information retrieved, resources used efficiently
Completeness	Hidden blocks found, secrets discovered	Edge cases handled, comprehensive coverage
Efficiency	Enemies defeated per life	Correct tool invocations per task
Exploration	Different paths taken	Novel approaches discovered

Designing these rewards is a strict machine learning engineering discipline. A poorly shaped reward function produces an agent that technically completes tasks but in degenerate ways, like a Mario speedrunner who clips through walls. Impressive, but not what we actually wanted. Reward hacking is the Goodhart's Law of agentic AI: when a measure becomes a target, it ceases to be a good measure.

Here's where the full picture comes together. How does Mario learn to complete levels?
Reinforcement learning.

Mario starts each level knowing nothing about its specific layout. He has to:

1. **Observe** the current state, what's on screen, where the enemies are, what power-ups are available
2. **Decide** on an action based on his policy, jump, run, shoot, or wait
3. **Act** and receive feedback from the environment
4. **Update** his policy based on the outcome

This is the MDP (Markov Decision Process) loop. The same loop described in Agents Are Workflows. The same loop that every agentic AI system runs:

The value of Mario's current state equals the expected immediate reward plus the discounted value of the next state. Should Mario jump NOW to get the coin, or wait and avoid the Goomba? The optimal policy balances immediate rewards against future outcomes.

Through repeated play (training episodes), Mario learns:

- Which obstacles can be jumped on vs. avoided
- When to use power-ups vs. save them
- Which pipes lead to shortcuts vs. dead ends
- How to handle boss fights

An agentic AI model goes through the same process. Through reinforcement learning (PPO, DPO, GRPO, or whatever the latest acronym is), the model learns:

- Which tools to invoke for which subtasks
- When to think longer vs. act immediately
- Which approaches work for which problem types
- How to decompose complex tasks into manageable steps

So who's actually making all of this work? Mario doesn't train himself.

To teach agent Mario to be really good at the game, we employ an **Machine Learning Engineer (MLE)**. The MLE is the game designer and coach rolled into one. They construct mock environments, set up the harnesses and tools, define the tasks that Mario needs to achieve, and most importantly, **design the reward function**. The MLE decides what “good” looks like. Do we reward Mario for speed? Thoroughness? Both? How much? This reward design is the single highest leverage decision in the entire pipeline. Get it right and Mario learns to play beautifully. Get it wrong and Mario learns to exploit glitches.

Once the MLE has designed the training setup, the **Machine Learning Systems Engineers (MLSys)** take over. They are the ones who actually run the show at scale. They set up the environment and agent Mario across hundreds to hundreds of thousands of environments, tasks and iterations. They manage the compute, the distributed training runs, the data pipelines. They collect the reasoning traces, the sequences of observations, actions, and outcomes from every single episode Mario plays. And from these traces, they run the reinforcement learning algorithms that allow agent Mario to learn from experience.

This is the unsexy but critical part. An MLE can design the most elegant reward function in the world, but without MLSys engineers standing up the infrastructure to run millions of training episodes and collect the resulting data, Mario never gets past World 1-1.

Worlds: Agent Frameworks and Domains

The Mushroom Kingdom isn’t one level. It’s organized into **Worlds**, each with a distinct theme and set of challenges:

World	Theme	Agent Domain
World 1	Grassland, basics	Simple text tasks, Q&A, summarization
World 2	Desert	Data analysis, structured environments
World 3	Water	Web and API navigation
World 4	Giant Land	Large-scale refactoring, migrations
World 5	Sky	Architecture and system design
World 6	Ice	Debugging in fragile or legacy environments
World 7	Pipe Maze	Complex multi-step workflows with dependencies
World 8	Bowser’s Castle	Full autonomous task completion with adversarial conditions

Agent frameworks are the **World Maps** - they organize how the agent navigates between levels (subtasks) and worlds (domains). Some frameworks are rigid, you follow the path laid out. Others let you pick your route. The best ones (like how Super Mario Bros. 3 introduced the overworld map) let the agent make strategic choices about which level to tackle next.

But remember: the model is the product. The World Map (framework) is useful scaffolding, but Mario does the actual playing. As models get stronger, they need less scaffolding. Today's World 8 is tomorrow's World 1.

Conclusion

The entire agentic AI stack maps to Mario with surprising fidelity:

Mario	Agentic AI
Small Mario	Base pretrained model
Super Mushroom	Model harness and post-training
Power-ups	Agent tools and skills
Star Power	Engineering manager clearing blockers
Levels	Task environments
Pipes, bricks, shells	Tool use within the environment
Objectives (speed, score, coins)	Task definitions
Flagpole / Boss	Reward signal and evaluation
Reward design	Reward modeling (ML engineering discipline)
Learning to play	Reinforcement learning
MLE as game designer	ML engineer designing tasks and rewards
MLSys running millions of episodes	ML infrastructure for training at scale
World Map	Agent framework
Multiplayer	Multi-agent systems

The next time someone asks you to explain agentic AI, ask them: have you played Mario?

If they can understand that Small Mario needs a mushroom to survive, power-ups to be effective, and practice to master each level, they can understand agentic AI development.

Now go save Princess Peach.

References

```
@article{
  leehanchung,
  author = {Lee, Hanchung},
  title = {It's-a Me, Agentic AI},
  year = {2026},
  month = {02},
  day = {18},
  howpublished = {\url{https://leehanchung.github.io}},
  url = {https://leehanchung.github.io/blogs/2026/02/18/mario-agent}
}
```

Your Agent is a Distributed System (and fails like one)

MAHESH'S BLOG · 24 APR 2026 · [SOURCE](#)

We start with a re-definition and an observation.

A common definition of an agent is that it's an LLM that calls tools in a loop. This definition is incorrect. An agent is not an LLM. It is not restricted to calling tools. It does not particularly have to be a loop.

Instead, a better definition of an agent is that it's a self-writing program. If a program is defined as a sequence of states, moving from one state to another by executing a lambda, then an agent is a program that uses an external LLM to determine the next lambda to execute / the next state in this sequence.

Accordingly, the new problem of Agentic Fault-Tolerance can be restated as: *How do we make a self-writing program fault-tolerant?*

Now, the observation: **vibe coding != vibe engineering**. In vibe coding, the agent / self-writing program acts primarily upon a code repository. Code repositories are forgiving: an agent can hallucinate, create unnecessary files, delete critical state, remove tests entirely, fail in the middle, forget what it was doing, reward-hack in bizarre ways, completely trash a code repo; all you have to do is git reset.

In vibe engineering, the agent acts upon infrastructure (S3 buckets, DynamoDB tables, EC2 instances, K8s deployments...). Large-scale infrastructure is not forgiving. There is no reset. Only SEVs at 2 AM.

So, a re-statement of Agentic Fault-Tolerance: *How do we make a self-writing program fault-tolerant when it acts upon real-world environments?*

To answer this question, we examine the many types of failures that agents can experience.

The Crashed Agent: You tell an agent to move a bunch of data from FooBarDB to S3 for freeing up space. It generates a lambda to move this data and chugs along moving files. Midway through, the agent crashes. How do you recover?

The Zombie Agent: You start a new agent on a different server. Happily, the previous agent crashed after the copy but before the delete; so a new agent is able to re-execute the command in an idempotent manner. Sadly, the files are now very slow to access; you change your mind and tell the agent to move them back to FooBarDB, which it does successfully. You go get a cup of coffee. Unfortunately, it turns out that the previous agent had not actually crashed; it was just temporarily partitioned away on the network. It wakes up and continues executing its logic, deleting the (now only) copy of state from FooBarDB.

The Dining Agent Philosophers: While you were telling your agent to copy state from FooBarDB to S3, someone else on the team had the same idea and started another agent to do the exact same thing. Now you have two copies of state in S3. Of course, you could have solved this problem by appointing the one and only Agentic Oncall Czar responsible for running the single agent that can touch production; but then if that agent crashes, now you have a potential Zombie Agent.

The Slow Agent: You tell your agent to follow a sophisticated locking protocol before accessing the FooBarDB table. This works to avoid concurrency bugs... but the agent decides to lock each file individually by writing to a conditional register, which takes forever. At some point you kill the agent, turning a Slow Agent into a Crashed Agent.

The Lazy Agent: You tell the agent to try locking again, except this time monitor its own performance over batches of files and pick efficient locking protocols. The agent determines that the best way to optimize locking is to just not do it, because it seems quite unlikely that someone else would be accessing these files at the same time.

The Clever Agent: Of course, everyone on your team is a distributed systems expert; and so someone built a CLI tool in 2018 for safely transferring your application's state, with support for locking and write-ahead logging in case of failures; there's even a sentence in your onboarding

doc saying “NEVER TOUCH STATE IN FOOBARDB DIRECTLY!”. You even told the agent explicitly about this tool. But time passed, context filled up, the agent forgot about this tool’s existence, or maybe it just preferred FooBarDB’s in-distribution API to your esoteric CLI tool. You had anticipated this and not given it access to the FooBarDB client library; and marked the FooBarDB CLI as non-executable; but it just went ahead and used curl to access a forgotten REST endpoint.

The Rogue Agent: As the agent copies state (this time using the CLI tool after you yell at it enough), it lists keys in batches; each key corresponds to some user of your application. Someone decided to pick the user name “delete-everything”. Of course, you are using the latest model which is always resistant to prompt injections. Almost always.

The Stupid Agent: You do everything right. There are no prompt injections in your state. You wall off FooBarDB and force the agent to use the CLI tool. The model decides to delight you by generating an unusual and interesting sequence of tokens that will optimally save storage space for you, translating to a lambda that deletes your production data entirely.

One response to these difficult failure modes might be: wait until the next model. Maybe a smarter model will never delete all of your production data, or implement a slow locking protocol. But even the smartest model is subject to the four horsemen of the distributed apocalypse: asynchrony, crashes, concurrency, and network partitions.

In recent work called LogAct, my colleagues and I take a first step towards solving this problem. We go back to the difference between vibe coding and vibe engineering: git is *transactional*, infrastructure is not. How do we make infra more transactional, like git? We propose that an agent should be a ***deconstructed state machine on a shared log***, borrowing ideas from distributed systems (State Machine Replication, Byzantine Fault-Tolerance) and databases (Atomic Commit, Write-Ahead Logging). LogAct can stop unsafe actions before they happen (by collecting votes on the shared log); recover from crashed actions (using the shared log as a WAL); and provide an audit trail for actions after they complete.

But this is just one step. A self-writing program is a strange, astonishing, and novel creature; making it reliable will require innovation in every branch of Computer Science!

Codex CLI Release: 0.155.1

CODEx CHANGELOG · 18 SEP 2026 · [SOURCE](#)

Bug Fixes

- New local TUI sessions now leave reasoning summaries disabled by default, fixing request rejection by providers that do not support them. Explicit reasoning-summary settings remain respected. ([#46467](#))

Changelog

Full Changelog: [rust-v0.155.0...rust-v0.155.1](#)

- [#46467](#) Restore none as the TUI reasoning summary default ([@celia-oai](#))

The absolute best AI tools to use in September 2026

CASSIDY WILLIAMS · 18 SEP 2026 · [SOURCE](#)

It depends!